Healthcare AI Platform

Architecture, Infrastructure & Deployment Blueprint

Target Environment: AWS (ap-south-1)
Orchestration: Amazon EKS
CI/CD: GitHub Actions

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.

1. System Overview & Tech Stack

The Healthcare AI Platform features a modern, distributed architecture built for high availability, security, and scalability. It streamlines clinical workloads through automated AI integrations while ensuring strict infrastructure segregation.

Layer	Technologies Used
Frontend	React
Backend API Engine	FastAPI
Asynchronous Worker & Queue	Celery / Custom Worker + Redis
Primary Database	Amazon RDS PostgreSQL (v16)
Artificial Intelligence / LLM	Ollama / OpenAI API Integration
CI/CD Pipeline	GitHub & GitHub Actions
Container Registry	Amazon Elastic Container Registry (ECR)
Container Orchestration	Amazon Elastic Kubernetes Service (EKS)

CI/CD & Deployment Workflow:

- Code changes are committed to the **GitHub** repository.
- **GitHub Actions** triggers automated builds and runs tests.
- Secure docker images are built and pushed to **Amazon ECR**.
- Application versions are seamlessly updated across **Amazon EKS Pods**.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.

2. Phased Infrastructure & Deployment Guide

Phase 1: AWS Infrastructure Foundations

The environment enforces network segregation using a multi-subnet topology across availability zones (AZs) with isolated routing rules.

Network Topology	Components & Gateway Attachments
VPC	Internet Gateway (IGW) attached for edge public communication
Public Subnets (A & B)	Houses NAT Gateways and Internet-facing Application Load Balancers (ALB)
Private Subnets (A & B)	Hosts protected EKS Worker Nodes, RDS PostgreSQL, and ElastiCache Redis instances

Step 1.1: Provision ECR Repositories

```
aws ecr create-repository --repository-name healthcare-backend
aws ecr create-repository --repository-name healthcare-frontend
aws ecr create-repository --repository-name healthcare-worker
```

Step 1.2: Provision Relational Database (RDS)

- **Engine:** PostgreSQL 16
  - **Deployment Model:** Multi-AZ (High Availability Architecture)
  - **Backups:** Automated Backups Enabled
  - **Network Location:** Strictly isolated within Private Subnets

Step 1.3: Bootstrap EKS Cluster via eksctl

```
eksctl create cluster \
--name healthcare-cluster \
--region ap-south-1 \
--nodes 3

# Verification Command
kubectl get nodes
```

Phase 2 & 3: Containerization & ECR Image Delivery

Dockerfiles are constructed independently for the Backend, Frontend, and Worker engines. Follow the workflow below to build, authenticate, and deliver images to ECR:

```
# 1. Authenticate local Docker daemon with AWS ECR Registry
aws ecr get-login-password --region ap-south-1 | docker login --username AWS
--password-stdin <account>.dkr.ecr.ap-south-1.amazonaws.com

# 2. Compile application container image
docker build -t healthcare-backend .

# 3. Tag standard image to target the ECR repository
docker tag healthcare-backend:latest
```


This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.

```
<account>.dkr.ecr.ap-south-1.amazonaws.com/healthcare-backend:latest

# 4. Push build artifacts securely to AWS ECR cloud repository
docker push <account>.dkr.ecr.ap-south-1.amazonaws.com/healthcare-backend:latest
```


This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.

Phase 4 & 5: Kubernetes Manifest Orchestration & CI/CD

All runtime state declarations are maintained dynamically inside Git-managed template manifests.

Component / Layer	Kubernetes Configuration & Execution Context
Namespace Configuration	kubectl apply -f namespace.yaml
Backend Core Engine	kubectl apply -f backend-deployment.yaml along with its associated Target Service Mesh.
Worker Orchestrator	Asynchronous task processing pods reading from the distributed queue runtime layer.
Distributed Cache Engine	Powered by **Amazon ElastiCache Redis** for rapid data retrieval and session offloading.
Relational Store Cluster	Maintained externally via native **Amazon RDS PostgreSQL** configurations.
Ingress Control Mesh	Provisioned via Helm to manage cluster access routing control natively: helm install aws-load-balancer-controller

Phase 5: GitHub Actions Automated CI/CD Engine

Pipelines are codified natively within the repository paths to completely automate testing, artifact compilation, registration, and rolling state updates in Kubernetes upon codebase modifications.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.

3. Enterprise Security, Resilience & Observability

Pillar	Architectural Implementation Details
Secrets Governance	**AWS Secrets Manager** centrally handles runtime sensitive variables, safeguarding configurations such as: DB_PASSWORD, JWT_SECRET, and OPENAI_API_KEY.
IAM & RBAC Control	Enforces strict granular access control using dedicated IAM Roles mapped to EKS Service Accounts for ECR registries, Secrets Manager vaults, and S3 buckets.
Data Encryption	**In-Transit:** TLS 1.2 strict policies termination at ALB HTTPS endpoints. **At-Rest:** Native RDS storage volume layer encryption via custom keys managed through **AWS KMS**.
Observability Mesh	**Metrics Engine:** helm install prometheus **Visual Dashboards:** helm install grafana **Distributed Tracing:** OpenTelemetry + Jaeger stack integration.
Audit Ledger Logs	Standard out streams and application runtime execution outputs are fed directly into **AWS CloudWatch Logs** for long-term retention and security analysis.
Resilience Architecture	**Compute Cluster (EKS):** Worker nodes span actively across 3 independent Availability Zones (AZ-1, AZ-2, AZ-3). **Database Layer:** Synchronous RDS Multi-AZ replication paired with highly available Redis Replication nodes.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.